
Important Note

Your time is important. We want you to attempt this assignment only if you meet the following selection criteria.

1. You are available for **7 hours during day time** on weekdays for this WFH internship.
2. You have a minimum of 6 months meaningful internship experience **with business organisations**. Please note that internships with professors / universities / college societies, and open source contributors will not be considered as internship experience. Personal projects are also not considered for your internship experience.
3. You are graduating from your college in 2027 or later.
4. Your college or university has **no objection** to you pursuing this internship.

Assignment for Internship - Backend

AI-Powered Transaction Processing Pipeline

Role	Backend + DevOps Intern
Level	0-2 years / Final year students
Time Limit	Submit within 4 days
Submission	GitHub repository link

1. Overview

You are given a CSV file of raw financial transactions exported from a fictional payment system. The data is intentionally dirty - mixed date formats, inconsistent casing, missing fields, duplicate rows, currency anomalies, and a handful of suspiciously large transactions. Build a backend API that accepts this CSV, processes it asynchronously through a job queue, uses an LLM to classify transactions and flag anomalies, and generates a structured summary report retrievable via a polling API.

2. What You Are Given

A file **transactions.csv** is attached with approximately 90 rows and the following columns:

Field	Value
txn_id	Unique transaction ID. Some rows have this blank.
date	Mixed formats: DD-MM-YYYY and YYYY/MM/DD both appear.
merchant	Merchant name - mostly clean.
amount	Numeric - some rows have a \$ prefix instead of a plain number.
currency	INR or USD - inconsistent casing (e.g. 'inr').
status	SUCCESS / FAILED / PENDING - inconsistent casing.
category	Spending category - some rows are blank.
account_id	Account reference.
notes	Free text - may say SUSPICIOUS, Duplicate?, or be empty.

3. Required Stack

- **API Framework:** FastAPI or Django REST Framework - your choice.
- **Database:** PostgreSQL.
- **Job Queue:** Celery + Redis or RQ + Redis.
- **LLM:** Any free-tier API - Gemini 1.5 Flash, OpenAI (if you have credits), or a local model via Ollama. No spend required.
- **Containerisation:** Docker and Docker Compose.

The entire system - API, worker, Redis, PostgreSQL - must start with a single **docker compose up** command. No manual setup steps allowed.

4. API Endpoints

POST /jobs/upload

Accept a CSV file upload. Validate it, create a Job record in the database with status=pending, enqueue the processing task, and return the job_id immediately.

GET /jobs/{job_id}/status

Return the current status of the job: pending, processing, completed, or failed. If completed, also include a summary field with high-level stats.

GET /jobs/{job_id}/results

Return the full structured output: cleaned transactions list, flagged anomalies, per-category spend breakdown, and the LLM-generated narrative summary.

GET /jobs

List all jobs with their status, filename, row count, and created_at timestamp. Supports filtering via ?status= query parameter.

5. The Processing Pipeline

When a job is dequeued, the worker must execute these steps in order:

a) Data Cleaning

Normalise date formats to ISO 8601. Strip currency symbols from amounts. Uppercase status values. Fill missing categories with 'Uncategorised'. Remove exact duplicate rows.

b) Anomaly Detection

Flag transactions where amount exceeds 3x the account's median as a statistical outlier. Flag rows where currency is USD but the merchant is a domestic-only brand such as Swiggy, Ola, or IRCTC.

c) LLM Classification

For transactions without a category, call the LLM to assign one of: Food, Shopping, Travel, Transport, Utilities, Cash Withdrawal, Entertainment, or Other. Batch your calls - do not make one LLM call per row.

d) LLM Narrative Summary

Make a single LLM call to produce a JSON summary: total spend by currency, top 3 merchants, anomaly count, a 2-3 sentence spending narrative, and a risk_level of low/medium/high. Store this as structured data.

e) Retry Logic

Retry failed LLM calls up to 3 times with exponential backoff. If all retries fail, mark that batch as llm_failed and continue - do not fail the entire job.

6. Suggested Data Model

You are free to design your own schema. As a starting point, consider:

Job

```
id, filename, status, row_count_raw, row_count_clean  
created_at, completed_at, error_message
```

Transaction

```
id, job_id (FK), txn_id, date, merchant, amount, currency
```

```
status, category, account_id, is_anomaly, anomaly_reason  
llm_category, llm_raw_response, llm_failed
```

JobSummary

```
id, job_id (FK), total_spend_inr, total_spend_usd  
top_merchants (JSON), anomaly_count, narrative, risk_level
```

7. Submission

Submit a public GitHub repository link. Include a README with setup instructions and example curl requests.

We will clone the repo, run **docker compose up**, and hit your endpoints directly.

3-Minute Technical Video Review (Required)

Submit a **maximum 3-minute video screen-share** (Loom/Zoom) walking through your submission. Treat this as an internal engineering design review with your Tech Lead.

Your video should have following :

1. System Design & Data Flow (~1 min)

- **The Blueprint:** Walk us through a high-level visual diagram (Miro, draw.io, or sketch) of your architecture. Do not just scroll through code.
- **The “Why”:** Explain the technical reasoning behind your choices (e.g., folder structure, database schema, specific libraries).
- **Request Lifecycle:** Trace the exact path a single request takes from the API endpoint to data persistence and back.

2. Bottlenecks & Scale (~2 mins)

- **The Breaking Point:** If application traffic scales by 100× tomorrow, where exactly will your current codebase break? (e.g., connection pools, memory, I/O?)
- **The Next Iteration:** If you had to re-engineer this for enterprise production to handle that scale, what specific architectural changes would you make, and what are their trade-offs?

Technical Review Video Rules

- **Time Limit:** Strictly under 3 minutes (evaluation stops at 3:00).
- **Visibility:** Camera must be turned **on**.
- **Access:** Ensure the video link has public view permissions enabled.

Submission Checklist

- ☐ GitHub repository link (public)
- ☐ High Level Visual Diagram (draw.io public)
- ☐ 3-minute technical video (public link or video file)